

Rootwell — Technical Guide: Stack, Dependencies, and Why

This document justifies every non-trivial technology choice in the repo — what it is, what problem it solves here specifically, what was rejected instead and why. It's written to be defensible in a technical interview, not just a changelog: each section ends with the trade-off, not just the pick.

0. Full service inventory

Service	Language / Framework	Key dependencies	Calls out to	Talks to Redis?
frontend	Next.js 14 (App Router) / React 18 / TypeScript	Tailwind CSS	gateway only	No
gateway	Python / FastAPI + Uvicorn	httpx	orchestrator	Yes (rate-limit counters only)
orchestrator	Python / FastAPI + Uvicorn	httpx, shared (cache, models)	all 7 agents + email	Yes (Tier 2, exclusively)
agent-intake	Python / FastAPI + Uvicorn	httpx, shared.llm (Anthropic)	knowledge-toxicology	No
agent-mapping	Python / FastAPI + Uvicorn	httpx, shared.llm (Anthropic)	knowledge-toxicology	No
agent-retrieval	migrating to Java	httpx (today)	knowledge-botanical, knowledge-compound	No
agent-safety	migrating to Java	httpx (today)	knowledge-rules	No
agent-scoring	migrating to Java	none — pure function	none	No
agent-explanation	Python / FastAPI + Uvicorn	shared.llm (Anthropic)	none	No
agent-reporting	migrating to Java	none — pure templating	none	No
email	migrating to Java	httpx (Resend REST API) today	Resend API	Yes (verification codes + rate limit)

Service	Language / Framework	Key dependencies	Calls out to	Talks to Redis?
knowledge-botanical	Java 21 / Spring Boot 3 (migrated)	spring-boot-starter-web, spring-boot-starter-data-redis	none	Yes (Tier 1, read-only)
knowledge-compound, -toxicology, -rules	migrating to Java	shared.cache (today)	none	Yes (Tier 1, read-only)
shared (Python library, not a service)	Python package	pydantic, redis, httpx, anthropic	—	—

The three agents that stay Python (`intake`, `mapping`, `explanation`) are exactly the three that `import shared.llm` — verified by grepping the repo for that import, not assumed from memory. Everything else is either already migrated or on the migration list (`ARCHITECTURE.md` §7).

1. Python backend stack

FastAPI + Uvicorn (ASGI)

What/why: FastAPI is a Python web framework built directly on ASGI (async), with request/response validation via Pydantic built in at the routing layer — a request body's shape is enforced before your handler code even runs, and the OpenAPI schema is generated for free from the same type annotations. Uvicorn is the ASGI server that actually runs it.

Why it fit this system specifically: every service in this pipeline spends almost all its time waiting on network I/O — an HTTP call to another service, a Redis round-trip, an Anthropic API call. That's the textbook case for `asyncio`: one thread can hold many in-flight requests open simultaneously because the GIL is released while waiting on I/O (see `mit-lincoln-lab-technical-qa.md` Q31/Q32 for the GIL/asyncio mechanics). A synchronous framework (Flask, classic Django) would need threads-per-request or a WSGI worker pool to get the same concurrency, burning more memory per in-flight request for no benefit, since none of this workload is CPU-bound.

Rejected alternative: Flask — simpler, more ubiquitous, but sync by default (would need `gevent`/threads bolted on to get comparable I/O concurrency) and no built-in request validation — every handler would hand-roll body parsing/validation that FastAPI gets from Pydantic for free.

Pydantic (`shared/shared/models.py`)

What/why: runtime data validation and serialization via Python type hints. Since there is no database (see §3), Pydantic models in `shared/shared/models.py` are, in the file's own words, "the closest thing this project has to a schema" — they're what a set of SQL `CREATE TABLE` statements would be in a conventional stack, except validated at the API boundary on every request instead of enforced by a database engine.

Trade-off acknowledged: this means schema consistency is a convention (every service that touches a `HerbRecord`-shaped dict agrees on its fields) rather than something a database enforces for you. That's acceptable at this scale (~13 services, one small team) and is exactly why the Java-side rewrite deliberately does *not* re-declare these as strict Java POJOs (see §5) — it avoids a second, Java-shaped copy of a schema that Python already owns, which would drift the moment one side changes.

`httpx` (async HTTP client)

Used for every service-to-service call. Chosen over `requests` specifically because `requests` is synchronous-only — using it inside an `async def` FastAPI handler would block the entire event loop for the duration of the call, defeating the point of using FastAPI at all. `httpx` has the same ergonomic API as `requests` but with a native `async` client.

Redis client: `redis` (redis-py) with `asyncio` support

`redis.asyncio` gives non-blocking Redis calls for the same reason `httpx` was chosen over `requests` — one blocking call anywhere in an `async def` handler stalls every other concurrent request on that worker. `shared/shared/cache.py` uses `scan_iter` (cursor-based, non-blocking iteration) rather than `KEYS` for the same reason a large-scale system would avoid `KEYS` in production: `KEYS` walks the entire keyspace in one blocking operation, and while today's ~80-record seed dataset makes that difference invisible, the code is written the way it would need to be at real scale rather than the way that happens to work today. (This is the same instinct behind Kafka consumer-group partitioning and Kubernetes HPA custom metrics discussed in `mit-lincoln-lab-technical-qa.md` — design for the access pattern, not the current data volume.)

Anthropic SDK, behind a mock-mode wrapper (`shared/shared/llm.py`)

What/why: `shared/shared/llm.py` is a two-function module: "is a real API key configured?" and "make the call." Every LLM-backed agent calls `llm.complete_or_none(...)`, which returns `None` in mock mode instead of a placeholder string — this forces every caller to have an explicit, real fallback path (see `agent-intake`'s keyword-matching `_mock_symptom_turn`, `agent-mapping`'s templated reasoning) rather than silently shipping fake-looking LLM output when no key is present.

Why this matters architecturally: it means "no external credentials configured" is a supported, tested runtime mode — not a degraded/broken state — which is what let this whole system be verified end-to-end with zero API keys. The alternative (making the LLM call required, erroring without a key) would make local development and CI impossible without provisioning real credentials, which is a worse trade for a system whose LLM calls are all advisory (worded generation), never authoritative (safety/scoring are structurally LLM-free — see `ARCHITECTURE.md` §5).

Model choice: Claude Sonnet 5 (`claude-sonnet-5`), overridable via `ANTHROPIC_MODEL`. The three call sites (intake NLU, mapping reasoning, explanation generation) are all short, low-token, latency-sensitive calls in a synchronous request path (the user is waiting on a chat reply) — a mid-tier model is the right cost/latency point; nothing here needs extended reasoning depth.

Why the LLM is never in the safety/scoring path

This is a defensible, explainable design constraint, not an accident: `agent-safety` and `agent-scoring` do not import `shared.llm` at all — a `grep` for the import proves it structurally, not just "we intended it that way." The reasoning: an LLM hallucinating "this herb is safe for you" is a real-harm failure mode in a way that a

hallucinated explanation sentence isn't. This maps directly to the human-in-the-loop / responsible-AI framing already on the resume ([PolicyMind](#)'s validation layer) — the same instinct, applied here as "keep the LLM downstream of every safety-relevant decision, never inside it."

2. Data layer: Redis-only, no database

Covered in depth in [ARCHITECTURE.md](#) §3 (the *what*); here's the *why*, stack-comparison style:

Rejected: PostgreSQL/any RDBMS. Would add schema migrations, a connection pool, and a second piece of local infrastructure for a dataset that's currently ~80 total records across 4 categories, entirely rebuildable from JSON in milliseconds, with zero relational query needs (every access is "get by id" or "list all of kind X"). The moment a real requirement needs relational querying (e.g., phase 2's live IMPPAT/ChEBI/ PubChem/CTD fallback, or genuinely complex herb-interaction queries), that's the trigger to add Postgres — not before. Introducing it now would be solving a scale problem that doesn't exist yet at the cost of real operational complexity today.

Rejected: MongoDB/a document store. Would remove the two-tier TTL model's simplicity (Tier 1 shared cache vs. Tier 2 per-session, hard-purge semantics) for no real gain — Redis's native key-TTL and [HSET/XADD](#) primitives already model exactly what's needed ([shared/shared/cache.py](#)), and a document store wouldn't add anything except another service to run.

Consistency model chosen: the whole system is one Redis instance — there's no distributed consistency problem to solve today (no multi-region replication, no eventual-consistency window). But the design already reflects the right instinct for when it would matter: Tier 1 (reference data) is read-heavy and tolerant of a few-hundred-ms staleness after a seed reload (availability-leaning), while Tier 2 (session state) needs the orchestrator's view to be immediately consistent with what it just wrote, since the very next request in the same session depends on it — the classic CAP-theorem framing from [mit-lincoln-lab-technical-qa.md](#) Q35, just not yet under an actual network partition because there's only one node.

3. Frontend: Next.js / React / TypeScript / Tailwind

Next.js (App Router) + React 18: server-rendered React with file-based routing — chosen for a chat-style app that's mostly one interactive page ([ChatPanel](#)) plus a handful of static content pages (About/Privacy/Terms). App Router's server components keep the static pages (which need no client-side interactivity) genuinely static, while [ChatPanel](#) opts into client-side rendering only where it needs state.

TypeScript: the frontend talks to the gateway over a JSON HTTP API with no shared-type-generation step (no OpenAPI-client codegen wired up yet) — TypeScript interfaces in [frontend/src/lib/api.ts](#) are hand-kept in sync with the Python/Java response shapes. This is a known manual-sync point; worth flagging honestly rather than presenting it as automatically safe.

Tailwind CSS: utility-first styling, chosen for iteration speed on a small number of components rather than a full design-system build-out — appropriate for a Phase 1 product, not a permanent architectural commitment.

Rejected: a separate SPA (Vite/CRA) + fully decoupled API. Next.js was chosen specifically because a few pages (About/Privacy/Terms) benefit from server rendering (simple content, good for the eventual public-launch SEO need) while the chat itself is a normal client-rendered app — Next.js lets both live in one project without maintaining two frontend build pipelines.

4. Containerization: Docker, multi-stage builds, Compose (not Kubernetes)

Docker, multi-stage builds: every service Dockerfile separates the build-time toolchain (pip installing from `requirements.txt`, or — on the Java side — `mvn package` against a full JDK+Maven image) from the runtime image (slim Python, or a JRE-only base for Java), so the final image that actually ships doesn't carry a build toolchain it never needs again. This is the direct analogue of the container-vs-VM reasoning in [mit-lincoln-lab-technical-qa.md](#) Q1 — containers share the host kernel and are only as heavy as what you actually put in the final layer, which multi-stage builds take advantage of deliberately.

Docker Compose, not Kubernetes, for local dev: this is a single-machine, single-environment local system — Compose gives per-service declarative config (env, ports, `depends_on` health-gating) with none of Kubernetes' operational surface (no cluster to run, no kubelet, no need for liveness/readiness *probes* as a Kubernetes primitive specifically — though see §5 for why the Java services still implement the readiness/liveness *concept* even without K8s to consume it). Compose is the right tool for "one dev, one machine, thirteen-plus containers that need to talk to each other by name" — Kubernetes would be solving problems (multi-node scheduling, rolling updates, horizontal autoscaling) this system doesn't have yet. [CLAUDE.md/ARCHITECTURE.md](#) §6 explicitly defers real cloud/Kubernetes deployment manifests to Phase 2, once there's an actual multi-instance, multi-environment need driving it — the same ECS vs. EKS judgment call from [mit-lincoln-lab-technical-qa.md](#) Q23 applies here: reach for the heavier tool only when its specific benefit (portability, existing K8s-ecosystem tooling like Istio) is actually needed, not by default.

No service mesh (Istio) here, and why that's the right call at this size: with ~13 services all inside one Compose network on one host, there's no multi-node traffic-routing problem, no need for canary traffic splitting, and mTLS between containers on one Docker bridge network isn't solving a real threat model the way it would across untrusted network boundaries. The gateway already does the one cross-cutting concern that matters at this scale (CORS + per-IP rate limiting) in application code. The honest scaling story: *if* this were deployed as genuinely multi-instance, multi-node services with a real internal trust boundary to enforce, Istio's sidecar-mTLS model ([mit-lincoln-lab-technical-qa.md](#) Q8–Q11) is exactly the next layer to reach for — pushing service-to-service identity/encryption out of application code and into infrastructure, uniformly, regardless of which of the two languages a given service is written in. That's a deliberate "not yet, but here's precisely when" rather than a gap.

5. Java backend stack (migrated services)

Spring Boot 3 (Java 21), Spring Web (blocking MVC, not WebFlux)

Why Spring Boot: it's the standard, most idiomatic choice for a Java REST microservice — auto-configuration, embedded servlet container (no separate app-server deploy step), a REST controller model (`@RestController`) that maps directly onto the same "one file, a handful of endpoints" shape the FastAPI services already have, which keeps the migration a faithful port rather than a redesign.

Why blocking MVC over WebFlux (reactive): the Python side chose `async` specifically because those services are I/O-bound fan-out callers juggling many concurrent in-flight requests cheaply. The migrated Java services so far ([knowledge-botanical](#)) are low-throughput, single-hop Redis lookups — there's no comparable concurrency pressure that would justify Reactor's steeper learning curve and harder-to-debug

stack traces. **This decision should be revisited when orchestrator and gateway migrate** — those two *do* fan out to many downstream calls concurrently today (`httpx.AsyncClient` in the Python version), which is exactly the shape WebFlux (or Java's virtual threads / `CompletableFuture` fan-out under plain Spring MVC) is built for. Noted here deliberately so it isn't decided by inertia when that migration slice comes up.

`spring-boot-starter-data-redis` (Lettuce client) over Jedis

Spring Boot's default Redis client is Lettuce (async-capable, netty-based, thread-safe connection sharing) — used here through the synchronous `StringRedisTemplate` API for the same reason blocking MVC was chosen: these are simple, low-concurrency lookup services today. Lettuce was kept (rather than swapping in Jedis, historically the "simple synchronous" choice) purely because it's the Spring Boot default with zero extra configuration — no reason to introduce a second Redis client library into the stack for a service this simple.

Cursor-based `SCAN` over `KEYS`: implemented via

`RedisConnection.keyCommands().scan(ScanOptions...)` rather than

`StringRedisTemplate.keys(pattern)` (which issues a blocking `KEYS` command). Direct parity with the Python side's `scan_iter` choice (§1) — same reasoning: `KEYS` walks the whole keyspace in one atomic, blocking op; `SCAN` pages through incrementally. Invisible at ~18 herb records, correct at any scale.

Jackson `Map<String, Object>` over a strict Herb POJO

Herb/compound/symptom/rule JSON is read as a generic `Map<String, Object>` rather than a hand-written Java class with declared fields. **Why:** the actual schema is owned by the Python side (`shared/shared/models.py`'s `HerbRecord`, etc.) and the seed loader (`seed/load_seed.py`) — a Java POJO here would be a second, independently-maintained copy of that schema that silently drops any field it doesn't know about the moment the Python side adds one. Since this service only ever filters (`linked_symptoms` contains X?) and passes records straight through, there's no correctness benefit to a strict type here, only a maintenance liability. **This is a deliberate "loose coupling over compile-time safety" trade specific to a pass-through lookup service** — it would be the wrong call in a service that actually constructs new domain objects or enforces invariants on herb data (there isn't one yet, but if one appears, that's where a real POJO earns its keep).

Correlation-ID logging (`CorrelationIdFilter`, MDC)

A `OncePerRequestFilter` reads/mints an `X-Correlation-Id` header, puts it in SLF4J's MDC for the request's duration, and echoes it in the response. This is the minimal version of the observability pattern in `mit-lincoln-lab-technical-qa.md` Q38 — "structured logging with a correlation/trace ID propagated across every service a request touches" — implemented at the single-service log-line level since there's no distributed tracing collector (Jaeger/Zipkin/OpenTelemetry backend) or metrics stack (Prometheus/Grafana) deployed for this local system yet. Explicitly the floor of that pattern, not the ceiling — propagating the header on *outbound* calls between services (so one correlation id threads through the whole `orchestrator` → `agents` fan-out) is the natural next step once more services are migrated and there's a real multi-service chain to trace.

Liveness vs. readiness (even without Kubernetes to consume it)

`GET /healthz` is kept as the existing contract (parity with every other service, checked by no automated prober today). The reasoning behind *also* separating liveness-style ("is my process up") from readiness-style ("can I actually serve a request right now," i.e., can I reach Redis) checks — per `mit-lincoln-lab-`

[technical-qa.md](#) Q4 — is worth internalizing even before there's a Kubernetes probe to wire up to it: the common mistake the Q&A guide calls out (one endpoint checking downstream health for *both* purposes) causes a transient Redis blip to make Kubernetes kill and restart every pod simultaneously the moment this *does* run on K8s. Building the distinction into the service now means the eventual Kubernetes migration is a manifest change (pointing [livenessProbe](#) and [readinessProbe](#) at two different existing endpoints), not a code change.

6. Security posture (current + what's deliberately deferred)

- **No authentication system.** Session possession ([session_id](#), a [uuid4\(\).hex](#)) is the capability — there's no account/login to authenticate against (see [ARCHITECTURE.md](#) §4). This is a scope decision for a stateless, anonymous-by-design app, not an oversight.
 - **CORS + per-IP rate limiting** happen in the gateway's application code ([services/gateway/main.py](#)), not at an infrastructure layer — appropriate at this scale (§4's Istio discussion applies symmetrically here: push this into a mesh/API-gateway-as-infrastructure only once there's a real multi-instance deployment to protect).
 - **No JWT/OAuth2 anywhere yet.** If/when this system needs real user accounts (e.g., a "save my history across sessions" feature explicitly deferred in [CLAUDE.md](#)'s Phase 2 list), the gateway is the natural place for token validation to live first ([mit-lincoln-lab-technical-qa.md](#) Q39/Q40 on JWT structure and OAuth2's relationship to it) — with service-to-service calls behind it staying trust-the-network-boundary until/unless a real multi-tenant or zero-trust requirement (Q43) justifies mTLS between every internal hop.
 - **OWASP-relevant choices already in place:** Pydantic/Jackson validation at every service boundary (rejects malformed input before it reaches business logic — mitigates injection-adjacent risks), no string-built Redis keys from unsanitized input ([_ref_key](#)/[HERB_PREFIX](#) are always code-controlled prefixes, user input is only ever the trailing id), and the email verification-code gate (rate-limited, TTL'd) specifically to prevent the export feature being abused as a spam relay.
-

7. Quick cross-reference to interview prep material

For rehearsing how to talk about these choices out loud, see [mit-lincoln-lab-technical-qa.md](#) in your interview prep folder ([Desktop/resume/](#)) — question numbers referenced inline above (Q1, Q4, Q8-11, Q23, Q31-32, Q35, Q38-40, Q43) map directly to sections of this guide. The pattern worth practicing: state the choice, state the one rejected alternative and why it lost, state the trade-off you're accepting today and the concrete trigger that would flip the decision later. That's the shape of a senior answer, not just "we used X."